

Animal Escape

1. Project Overview

Project Name: Animal Escape

Concept: An action-arcade 2D survival game where the player controls an "animal" navigating through a maze to collect items for points. The player must manage health (HP) and stamina (Mana) while evading a highly intelligent "Wolf" (AI Bot) that utilizes pathfinding and predictive movement to hunt the player down.

2. Project Review

Inspiration: Pac-Man

Improvements & Enhancements: This project evolves the classic maze-collection mechanic by introducing resource management (HP/Mana bars), a character selection system with unique stats, diverse tactical items (Traps, Shield, Wall Phasing, Clock, Heal), and advanced AI behavior that predicts player movement rather than just following a set path.

3. Programming Development

3.1 Game Concept

- **Mechanics:** Players select one of three unique characters to enter the maze. Players move using Arrow keys or WASD and can hold 'Shift' to dash (consuming Mana). The objective is to achieve the highest score possible.
- **Difficulty Scaling:** For every 10 points earned, enemy speed increases, and the spawn rate of environmental traps increases.
- **Combat & Survival:** Collision with the enemy deals 50 damage (reduced to 25 if a Shield is active). If HP falls below 50, a low-health red flashing effect appears on the screen.

3.2 Object-Oriented Programming Implementation

1. **Player Class** Manages the state and behavior of the player character.

- **Attributes:** HP, Mana, Speed, CharacterType, isShielded, Position(x, y)
- **Methods:**
 - move(): Handles basic movement input.
 - dash(): Executes a quick burst of movement.
 - takeDamage(): Calculates health reduction and invulnerability frames.
 - useItem(): Triggers the effect of a gathered item.
 - checkBoundaries(MapManager): Interfaces with the MapManager to validate movement within permitted bounds.

2. **Enemy Class** Handles AI logic and pursuit mechanics.

- **Attributes:** Speed, State (Hunting/Stunned), TargetPosition(x, y)
- **Methods:**
 - calculateAStarPath(MapManager): Performs pathfinding by accessing MapManager.GridData to identify traversable nodes and obstacles, ensuring the Enemy dynamically recalculates the most efficient route to the Player's current position.
 - predictPlayerPath(Player): Uses player position and velocity to calculate Predictive AI movements.
 - patrol(): Moves along a predefined path when the player is out of range.
 - attack(Player): Executes offensive actions when within range of the player.

3. **Item Class (Base Class)** The parent class for all in-game collectables.

- **Attributes:** ItemType, SpawnRate, Duration, Position(x, y)
- **Methods:**
 - spawn(): Handles the instantiation of the item in the game world.
 - onCollect(Player): An abstract method using Polymorphism to define unique effects when a player picks up a specific item.
 - respawnTimer(): Manages the cooldown period before the item reappears.

4. **MapManager Class** Manages map structure and collision detection.
 - **Attributes:**
 - MapID: Identifier for the currently active map.
 - GridData[][]: A pre-defined 2D Array representing the manual layout (0 for walkable paths, 1 for stone walls), ensuring consistent level design for data analysis.
 - WallCoordinates[]: A list of coordinates derived from the grid to render static obstacles in Pygame.
 - **Methods:**
 - loadMap(): Initializes the game environment by reading hand-crafted layout data stored within the system.
 - isWalkable(x, y): Validates if the player's next coordinate matches a "0" in the GridData
 - resolveOverlap(Player): Contains the "Push Algorithm" to prevent the player from clipping through walls by pushing them back into valid space.
5. **StatsLogger Class** Handles data logging to CSV files.
 - **Attributes:** LogBuffer, FileName, RecordingInterval
 - **Methods:**
 - logEvent(data): Captures specific gameplay events.
 - saveToCSV(): Exports buffered data to a physical CSV file.
 - autoRecord(Player, Enemy): A combined Event-driven and Periodic system to continuously log player and enemy metrics.

3.3 Algorithms Involved

- **A Pathfinding:*** Calculates the shortest traversable path from the enemy to the player while avoiding obstacles.
- **Predictive AI Logic:** Calculates the player's projected position 2-3 seconds into the future based on current velocity to allow the enemy to "cut off" the player.
- **Collision Response (Push Algorithm):** A safety logic that detects if a player is overlapping with a wall (e.g., after the Wall-Phase item expires) and pushes them to the nearest empty coordinate.

4. Statistical Data (Prop Stats)

4.1 Data Features

Feature	Why is it good / What can it be used for?	How will you obtain 100 values of this feature data?	Which variable (and class)?	How will you display this data?
Item Collection	To analyze item popularity and determine if certain items are overused or underused for game balancing.	Log data every time the player collects an item during gameplay.	ItemType (Item class / StatsLogger)	Pie Chart showing percentage of each item collected
Player Position & Enemy Position	To identify high-risk zones, common paths, and bottleneck areas in the map.	Record player position (x, y) every 1 second and distance to enemy when within 100 pixels.	x,y (Player class) / Distance (Enemy+ Player class).	Bar Chart (Frequency per Zone) and Bar Chart (Average Distance).
Mana Usage	To analyze stamina management efficiency and how it affects survival time.	Record the player's mana value every 1 second during gameplay (especially while dashing).	Mana (Player class)	Line Graph (Time vs Remaining Mana)
Combat & Damage	To study how quickly the player loses health and evaluate game difficulty.	Log HP and timestamp every time the player receives damage.	HP, Timestamp (Player class)	Area Chart (Time vs HP)
Enemy Proximity	To evaluate game balance by analyzing how close the enemy gets to the player during gameplay.	Record distance between player and enemy whenever the enemy is within 100 pixels.	Distance, Score (Enemy + Player class)	Bar Chart (Character Type vs Average Distance)

4.2 Data Recording Method

Statistical data will be recorded and saved into a **CSV file (stats.csv)**

4.3 Data Analysis Report

Feature Name	Graph objective	Graph type	X-axis / Sections	Y-axis
Item Collection	To visualize the distribution of item popularity and identify if certain items are over/under-utilized.	Pie Chart	Item Types (Heal, Trap, Shield, etc.)	Percentage (%) of Total Collected
Player Position & Enemy Position	To identify high-risk zones or bottleneck areas by measuring the frequency of presence in specific map sectors.	Bar Chart	Map Zones (e.g., Zone A, B, C,D)	Frequency of Presence (Count).
Mana Usage	To analyze the efficiency of stamina management and the rate of depletion versus survival time.	Line	Survival Time (Seconds)	GraphRemaining Mana Level
Combat & Damage	To illustrate health loss progression over time and analyze the game's difficulty curve.	Stacked Area Chart	Time (Timestamp)	Current HP (Cumulative Loss)
Enemy Proximity	To compare game balance by assessing how close different characters stay to the enemy versus their score.	Bar Chart	Character Types (3 unique characters)	Average Distance from Enemy (px)

Statistic table

Feature Name	Statistic value
Survival Time, Remaining Mana Enemy Distance, Player Score	Mean, Median, Max, Min

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal Submission: Finalize game concept and design the initial UML Class Diagram.
2	9	Core Engine Development: Implement Player movement, Map system , and Basic Collision detection.
3	10	Systems & Mechanics: Add the Item system (6 types), Character status (HP/Mana), and UI overlays.
4	11	AI & Data Integration: Implement Predictive AI logic and the StatsLogger (CSV recording system).
5	12	Data Collection Phase: Playtesting to gather 100+ event records and Bug fixing.
6	13	Final Submission (Draft): Generate data visualizations (Graphs/Charts), prepare the Final Report, and record the Video Presentation.
7	14	Submission week

6. Document version

Version: 3.0

Date: 31 March 2025

Editor: **Chatthaya Tipatnaranan 6810545531**

Date	Name	Feedback
14/03	Peraya	Major Changes Needed - Missing Class Details (Section 3.2): You listed 5 classes, but you failed to provide the attributes and methods for each. A name alone does not prove OOP design. I need to see how Player interacts with MapManager and how Enemy uses the A* algorithm internally. - Data Feature Feasibility (Section 4.1): You are at high risk of failing the "100 records per feature" rule. Specifically, "Skill Activation" and "Item Collection" are event-based. Unless a player picks up 100 items in a single session (unlikely), you won't meet the requirement per feature. You may redefine these to be high-frequency (e.g., logging player (x, y) coordinates or distance_to_enemy every 0.5 seconds). Minor Changes Needed - Data Consistency: Section 4.3 mentions a "Character Selection" chart, but "Character Selection" is not listed as one of your 5 measurable features in Section 4.1. Your analysis must match your data collection exactly. - 4.1 Formatting: Remove the visualization descriptions from Section 4.1. Focus only on what is being measured and how often. But don't be sad overall the game concept is strong, the technical documentation lacks the specific depth required for a development-ready proposal. Keep going na 🙌🙌
14/03	Techaphat r	Note: Major and Minor Changes you can base from Peraya's comments. Your game concept is interesting but there's some part that can be a bottleneck with a limited time: Testing, and AI Prediction. It doesn't mean that you can't do the way you write, just manage your time carefully. Good luck with your work kub. (if you have any question you can ask us directly in our discord)

30/03	Peraya	- You have a Graph Plan and a Feature Table, but you are missing the Table of Statistical Values (showing Mean, Median, Max, Min, etc.). The rubric requires at least 1 table specifically for showing statistical values. - You also use a Heatmap for coordinate-based distribution data (Player Position) which it may not use, Heatmap should be used for relation data.
-------	--------	---